

Guidance, Navigation, and Controls (GNC)

Overview

The GNC subteam designs the ROV's "brains" – the computing, sensors, and control logic that steer the vehicle. In Pioneer-class ROVs (introductory college level), teams typically rely on teleoperation with feedback (e.g. depth or orientation hold) rather than full autonomy. Common tasks include stable depth control, heading control, and proportional thruster mixing. Teams often start with manual joystick control and may implement simple PID loops (e.g. maintain level or depth using the IMU/depth sensor data). Because Pioneer ROVs use relatively few thrusters (often 4: two horizontal, two vertical), control is simpler than higher classes but still benefits from well-tuned code and reliable communications.

Control System Hardware

- **Single-Board Computer (SBC):** Most teams use a Linux SBC to host the control software. The Raspberry Pi 4 is very common (especially with Blue Robotics' ecosystem) ¹. For example, Blue's Navigator flight-controller "hat" requires a Raspberry Pi 4 ¹. Other options include NVIDIA Jetson (for heavy vision tasks) or small PCs, but Pi4 is popular due to cost and support. The SBC typically runs Raspbian/Ubuntu (often via a distribution like BlueOS ²) and communicates over Ethernet or USB tether to the surface station.
- **Control Board / Autopilot:** Pioneer teams often use a kit-provided control box (e.g. SeaMATE's Barracuda/EagleRay kits) or a prebuilt autopilot board. The SeaMATE Barracuda kit, for instance, includes a control box with an Arduino Mega for I/O and motor outputs ³. Alternative approaches are popular too: many teams use flight-controller boards (like Pixhawk or Blue Robotics' **Navigator**) running the ArduSub (ArduPilot ROV) firmware. For example, Blue Robotics' standard setup is a Raspberry Pi (running BlueOS) paired with a Navigator running ArduSub, controlled via QGroundControl or the BlueOS Cockpit interface ⁴. In one build, a Pixhawk autopilot handled low-level control (IMU, depth, magnetometer) while a companion Raspberry Pi streamed video and ran higher-level logic ⁵. In contrast, the Barracuda kit's Arduino-based control board simply provides PWM signals to thruster ESCs under LabVIEW or Arduino code. Overall, using an autopilot board (Pixhawk/Navigator) is increasingly common because it integrates sensors and mixing, but many first-year teams use the kit's Arduino control board or a similar microcontroller as a simpler option ³ ⁴.

Figure: Blue Robotics Navigator flight controller (for Raspberry Pi). It provides an IMU, dual magnetometers, barometer, ADCs and 16 PWM outputs ⁶ ⁷. - **Thrusters and ESCs:** Brushless DC thrusters are driven by ESCs. The industry-standard choice is the Blue Robotics T200 thruster (110 W) which is used on thousands of ROVs worldwide ⁸. Pioneer teams typically equip their ROV with 4 thrusters: two mounted for horizontal (forward/back) motion and two for vertical (up/down) control ⁹. For example, one team's 4-T200 setup allowed full 3D maneuvering (two fore/aft, two vertical) ⁹. Operating these thrusters requires 30–40 A ESCs and a power supply (e.g. 12–48 V battery) ¹⁰. The ESC receives a PWM signal (often from the

control board or autopilot) to regulate motor speed. In practice, the control code mixes joystick inputs or autonomous commands into 4 PWM outputs for the ESCs.

- **Power and Tether:** Pioneer ROVs typically use a single high-voltage power line (12 V or 48 V) and a separate data tether. Tethers often carry Ethernet signals (via tether-specific transceivers like Fathom-X ¹¹) plus power. Teams must include safety features (like a main switch and fuse on the tether) and strain relief. (The 2025 Pioneer rules even allow 12 V or 48 V power systems, per the SeaMATE kit update.)

Sensor Suite

- **Inertial and Heading Sensors:** Every GNC system needs an IMU (3-axis accelerometer + gyro) for orientation feedback. Most autopilots/flight controllers provide this. A 3-axis magnetometer (compass) is also common to maintain heading. For example, Blue's Navigator includes a 6-DOF IMU plus dual 3-DOF magnetometers ⁷. In the Pixhawk/ArduSub setups, the autopilot's IMU is fused (often via a Madgwick or Kalman filter) to estimate attitude; the external magnetometer gives heading ⁵.
- **Depth Sensor:** Depth is measured with a pressure sensor. Common sensors include the MS5837 family (used by Blue Robotics Bar30 or Bar02 depth modules) ¹². Teams use this feedback to hold constant depth or perform depth-based tasks. (No GPS underwater, so the ROV uses depth + heading/IMU to gauge position.)
- **Cameras:** Teleoperation relies on video. Pioneer-class ROVs usually have at least one forward camera (and often a downward camera). Any standard USB camera can work; many teams use H.264-capable webcams or Raspberry Pi cameras. For instance, open-source ROV code often assumes an H.264 USB camera ¹². The video stream is sent over the tether to the surface and displayed to the driver (often via GStreamer or the ground-station software).
- **Auxiliary Sensors:** Depending on mission needs, teams may include additional sensors. Leak detectors (gold probes connected to the ROV) are common for safety. Temperature or conductivity sensors can be used if tasks require environmental data. High-end ROVs might add altimeters or even sonar for navigation, but Pioneer teams typically focus on cameras, IMU, compass, and depth.

Software: Languages, Frameworks, and Tools

- **Programming Languages:** Teams most often program in **Python** or **C++**. Python is favored for fast development (e.g. camera processing with OpenCV) and is used by many ROS-based teams. C++ is used for performance-critical code (ROS core nodes, autopilot libraries). Some teams have even experimented with **Rust**; for example, an open GitHub project for the 2025 ROV uses Rust and the Bevy engine to control a Pi-based ROV ¹³ ¹². (However, Python and C++ remain far more common.) MATLAB or Simulink might be used in research, but less in Pioneer-class due to cost/licensing. **LabVIEW** (graphical programming) is also used by beginners: MATE provides LabVIEW+Arduino examples ¹⁴, and some teams use NI MyRIO or LabVIEW code for the Barracuda kit's Arduino controller ³ ¹⁵.

- **Robotics Frameworks / OS:** Many teams leverage robotics middleware. **ROS (Robot Operating System)** is popular: it provides drivers (e.g. joystick, camera), messaging, and ecosystem packages. Wentworth's 2024 ROV code is structured as a ROS workspace and is ~94% Python ¹⁶ ¹⁷. Case Western Reserve's ROV "bootcamp" uses **ROS 2** in Python (with PyQt and OpenCV) for driving exercises ¹⁸. Even if teams don't use ROS, Blue Robotics' **BlueOS** (a Raspbian Linux image) is common: it bundles ArduPilot, streaming services, and can run onboard Python/C++ code ².

On the surface station, the software typically runs on Linux or Windows. ROS code usually runs on Ubuntu (with ROS Noetic or ROS2 Foxy/Humble). LabVIEW runs on Windows. *QGroundControl* (cross-platform) is often used for tuning ArduSub and viewing telemetry. Blue Robotics' *Cockpit* app can also display video and sends joystick commands to the ROV.

- **Development Environment:** A good IDE/editor helps productivity. **Visual Studio Code** is widely recommended (with extensions for Python, C++, ROS) ¹⁹. Teams have also used CLion or VS2019 for C++, PyCharm for Python, etc. Version control (Git/GitHub) is essential – many teams share code on GitHub. For LabVIEW, National Instruments' LabVIEW IDE is used. It's advisable to work in a 64-bit OS to support driver software and multiple cameras.
- **Key Libraries and Tools:** Common libraries include OpenCV (Python/C++) for vision, MAVLink for autopilot communication, and message protocols for sending commands. For camera capture and streaming, GStreamer is often used in Linux; some teams also use VLC or custom capture code. The code should run at ~10–30 Hz control loops (IMU reading, thruster outputs). MATE provides example libraries for the Arduino kit and for reading sensors, which teams can adapt.
- **Example Open-Source Code:** There are active GitHub repos to study. For example, the *Eoghanmc22/mate-rov-2025* repo provides a full ROV stack in Rust using Pi4/Blue Navigator and common sensors ¹³ ¹². Wentworth's *WIT-IEEE-MATE-ROV* repos contain C++ and Python code for the 2024-25 ROV (including a ROS workspace) ¹⁶ ¹⁷. Case Western's *rov-software-bootcamp* teaches ROS2 and PyQt ¹⁸ ¹⁹. Teams should review these for guidance on structure and best practices.

Sensors and Data Integration

- **Sensor Fusion:** Guidance/navigation often combine multiple sensors. For example, depth hold might use a PID loop on the pressure sensor data, while heading hold uses compass+IMU. An IMU by itself drifts over time, so combining it with magnetometer helps stabilize yaw. If using an autopilot (ArduSub), it automatically fuses IMU+magnetometer (with Madgwick/Complementary filter) for attitude estimation. Otherwise, implement your own filters or simple complementary filter to merge accelerometer/gyro.
- **Video Processing:** Real-time tasks (like target detection) can be done onboard (e.g. using Python/OpenCV on the Pi) or on the surface if bandwidth allows. Many Pioneer teams stream raw video and do processing on the surface station. Regardless, the ROV software must encode and send frames efficiently (BlueOS and ROS provide utilities for this).
- **Data Protocols:** If using ArduSub, the standard is *MAVLink* protocol for commands and telemetry. Otherwise, teams often write TCP/UDP sockets or use ROS topics (over ROS network) to send

commands to thrusters and receive sensor data. Ensuring reliable low-latency communication over the tether is crucial.

Best Practices

- **Version Control and Documentation:** Use Git and comment your code. Maintain clear documentation (block diagrams of electronics, calibration steps, code README, etc.). Other teams' technical reports (often published on MATE's site) are valuable references.
- **Modular Architecture:** Separate low-level drivers (reading sensors, writing PWM) from high-level logic (joystick mapping, control loops). For example, have one module read IMU/depth and compute orientation, and another take those values to compute thruster commands. This makes testing easier.
- **Testing and Calibration:** Test components individually before pool runs. Calibrate the IMU (level it) and compass (in-air calibration) early. Ensure ESCs are calibrated (sometimes by powering the ROV out-of-water briefly). Verify thrusters spin in the correct direction (CW vs CCW props). Dry-run joystick control on a bench: confirm that pushing forward makes the ROV go forward, etc. Also test computer vision and ensure the camera orientation matches the code's assumptions.
- **Safety and Reliability:** Implement a kill-switch (hardware or software) to disable thrusters in an emergency. Check tether and connectors for leaks. Include an LED indicator or buzzer for power warnings. Use strain relief on cables to avoid shorts. For Pioneer class, simplicity and robustness beat overengineering.
- **Surface Station Setup:** The surface computer should run whatever UI you build (e.g. custom Python GUI or ROS RViz/Qt interface) and/or QGroundControl. A standard gamepad (Xbox/PS controller) is ideal for driving – most teams use one. Ensure the computing OS has drivers for the controller and cameras. Keep your GUI responsive: e.g. display two video feeds (forward/down) at reasonable frame rates.
- **Competition Rules Compliance:** Pioneer class mandates certain constraints (e.g. tether spec, buoyancy volume, safety features, and use of approved control systems). If using a SeaMATE kit, follow their wiring and software guidelines. If you deviate (custom PCBs, novel sensors), make sure it's allowed by the Pioneer manual.
- **Review and Iterate:** Allocate time to refine your control algorithms. For example, a P-controller for depth may need tuning of gain. Plot logged sensor data (IMU, depth) to analyze noise and drift. Peer-review code and hardware wiring. Attend MATE workshops or community forums (e.g. Blue Robotics forum, ROV Discords) to ask questions early.

Common Choices (Summary)

- **Programming:** Python and/or C++ are standard. Python is widely used (e.g. ROS nodes, OpenCV) ¹⁷
¹⁸ . C++ may be used for performance-critical code or complete ROS nodes. LabVIEW (with Arduino)

is also supported by MATE for Pioneer teams ¹⁴ ¹⁵ . Emerging languages like Rust have projects but are optional.

- **IDE:** Visual Studio Code is a popular free editor (teams recommend it) ¹⁹ . Other common tools are VSCode, CLion (C++), PyCharm, or the LabVIEW IDE (if using LabVIEW). Use version control integration (e.g. Git) to collaborate.
- **Control Boards:** Prebuilt kits (Barracuda/EagleRay) come with a control enclosure and Arduino. Alternative modern choices include Pixhawk/PX4 boards or Blue's Navigator. Autopilot-based setups (ArduSub) are common and robust ⁴ . Custom microcontroller boards (Arduino Mega or Teensy) are also possible for simple vehicles ³ .
- **OS/Framework:** Linux is the norm for in-vehicle code. BlueOS (Raspbian-based) is provided by Blue Robotics ² . ROS1 (Noetic) or ROS2 can be used on Ubuntu if desired. LabVIEW will run on Windows for those teams using MATE's examples ¹⁵ .
- **Sensors:** Typical suite is forward camera + down camera, a 6-DOF IMU, 3-DOF magnetometer, and a pressure/depth sensor. (Some ROVs also carry additional sensors like sonars or manipulators for tasks, but these are mission-specific.) Always include a leak sensor. Blue's Bar30 depth sensor and Navigator IMU/mag are examples of common hardware ¹² ⁵ .

Sources: Technical reports and code from recent MATE ROV teams; Blue Robotics documentation; MATE ROV guides. For example, an ROV using Blue equipment ran BlueOS/ArduSub on a Pi4 and Navigator with QGroundControl ⁴ , while an educational ROV used an Arduino Mega with LabVIEW control ³ ¹⁵ . Open-source teams publish their code on GitHub, such as *eoghanmc22/mate-rov-2025* (Rust/Bevy stack) ¹³ ¹² and WIT's ROV repos (ROS/Python) ¹⁶ ¹⁷ , which can be studied for guidance. These examples, along with MATE's own modules and manuals, embody current best practices for Pioneer-class GNC.

1 4 11 Using Blue Robotics Components for MATE ROV Competition - MATE ROV - Blue Robotics Community Forums

<https://discuss.bluerobotics.com/t/using-blue-robotics-components-for-mate-rov-competition/17152>

2 6 7 Navigator Flight Controller for Raspberry Pi 4 for ROVs, robots, and drones

<https://bluerobotics.com/store/comm-control-power/control/navigator/>

3 9 15 2024 teachnical documentation NESR

<https://20693798.fs1.hubspotusercontent-na1.net/hubfs/20693798/PN09%202024%20teachnical%20documentation%20NESR.pdf>

5 Final Report

<https://kastner.ucsd.edu/wp-content/uploads/2025/06/admin/tryton.pdf>

8 T200 Thruster: ROV thruster for marine robotics propulsion

<https://bluerobotics.com/store/thrusters/t100-t200-thrusters/t200-thruster-r2-rp/>

10 Thruster User Guide

<https://bluerobotics.com/learn/thruster-usage-guide/>

12 13 GitHub - Eoghanmc22/mate-rov-2025

<https://github.com/eoghanmc22/mate-rov-2025>

14 MATE Module 11: Computer Control: Intro to Programming

<https://www.merlot.org/merlot/viewMaterial.htm?id=1214511>

16 17 GitHub - WIT-IEEE-MATE-ROV/wurov2: 2024 MATE ROV Competition Code

<https://github.com/WIT-IEEE-MATE-ROV/wurov2>

18 19 GitHub - CWRUbotix/rov-software-bootcamp: Software bootcamp for MATE ROV

<https://github.com/CWRUbotix/rov-software-bootcamp>